



Sentiment Analysis

- Inder Singh



Agenda

- Defining Sentiments
- Defining Sentiment analysis
- Sentiment analysis Tools
- Use case: Understanding bitcoin price using twitter sentiments
- Q&A

Defining Sentiments

- Sensitiveness to a topic
- Can be expressed as:
 - ✓ Opinions
 - ✓ Attitudes
 - ✓ Emotions
- Subjective vs Objective
- Categories of Sentiment expressions:
 - ✓ Sarcasms – *Lost coins only make everyone else's coins worth slightly more. Think of it as a donation to everyone.*
 - ✓ Emojis – *Prices are shooting up 😊 vs Prices are shooting up ☹*
 - ✓ Negation – *I do not disagree that Brock Pierce said Bitcoin is not a currency as it's too expensive to actually use in transaction*
 - ✓ Adverbial – *Sometimes people really lose money in volatile market*
 - ✓ Conflicting attitudes – *Ethereum has better return than bitcoin but bitcoin is more predictable*
 - ✓ Local dialects and slangs – *Prices ding-dang with launch of DMB*
 - ✓ Negative words used in positive sense – *Up or down the Bitcoin spurs us on the torture. Interesting how quickly money can be made*

Defining Sentiment analysis

- Sentiment analysis aka Opinion mining is a type of data mining:
 - Facebook Posts
 - Twitter
 - Expedia
 - Instagram
 - Product review websites
 - Direct customer feedback
- Using NLP, statistics, or machine learning methods to extract, identify, or otherwise characterize the sentiment content of a subjective information
- Sentiments are expressed in Polarity and can be Positive, Negative and Neutral.

Tweet	Sentiment
Crypto Activist Andreas Antonopoulos Calls Bitcoin ETFs a Terrible Idea UNHASHED https://t.co/YUUigAUm5m Crypto Bi	-1
Bitcoin has been one of the worst speculative bets of the year investment	-0.75
Bitcoin scams on the rise as fraudsters tempt victims with fake celebrity endorsements	-0.5
SOCIALCHAINS Bitcoin The project is becoming more popular every day Excellent performance to run	0.6375
Despite price crash believers say the best is yet to come	1

- Sentiment analysis has been used for:
 - Customer management - To manage customer expectation
 - Competitive advantage
 - Public opinion towards policy announcements by Obama govt.
 - Product reviews
 - Finance

Sentiment Analysis Tools

NLTK Sentiment Analyzer

- based on ML approaches
- vectorizes words into Positive or negative
- words not appearing in lexicon is neutral.
- support for Polarity based liu hu lexicon
- support for intensity based vectors such as VADER sentiments

TextBlob

- Text processing python library
- Returns tuple of different types of sensitivity (Polarity, subjectivity)
- Polarity is between -1.0 to 1.0
- Subjectivity between 0.0 to 1.0

Sentiment Analysis Tools

Scikit-Learn

- Simple yet powerful library available in python
- Used for data mining and data analysis
- Supports algorithms such as clustering, regression, classification etc.
- Sentiment analysis can be performed by training and testing your model

Stanford coreNLP

- Set of Lexical analysis tool, algorithm is based on sentences rather than words
- Sentiment analysis treebank has sentiment labels for over 200k phrases
- Has parse tree of 12k sentences, thus posing new challenges for sentiment compositionally
- Uses RNNs
- Can be used for:
 - Grammatical analysis
 - Text Analysis
- Supports:
 - API
 - web service
 - various languages



Use case: Understanding bitcoin price with historical tweets sentiments

Prerequisites

1. Twitter developer account
2. Knowledge of Python

Steps

- Data Extraction:
 - Get historical tweets between range of dates
 - Get Historical bitcoin price for the same range of dates
- Data Purging:
 - Clean up the tweets to transform it to meaningful texts
- Data Storage:
 - Store the data for visualization to a csv
- Sentiment Analysis
 - Plot Graph for analysis
- Data Visualization and Analysis

Data Extraction - Historical

➤ Setting up tweepy

```
auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
```

```
api = tweepy.API(auth, wait_on_rate_limit=True)
```

➤ Calling the tweepy cursor

```
for tweet in tweepy.Cursor(api.search, q="#Bitcoin", count=100,
                           lang="en",
                           since="2018-08-13", until="2018-08-14").items():
```

➤ Getting Bitcoin Price

```
# *****Function to get historical tick prices from
crypto compare api*****

def hourly_price_historical(symbol, comparison_symbol, limit,
                             aggregate, exchange=''):
    url = 'https://min-
api.cryptocompare.com/data/histohour?fsym={} &tsym={} &limit={} &agg
regate={}\'
        .format(symbol.upper(), comparison_symbol.upper(),
limit, aggregate)
    if exchange:
        url += '&e={}'.format(exchange)
    page = requests.get(url)
    data = page.json()['Data']
    df = pd.DataFrame(data)
    df['timestamp'] = [datetime.fromtimestamp(d) for d in
df.time]
    return df

#
*****
```

```
import pickle
import quandl
import plotly.offline as py
import plotly.graph_objs as go

py.init_notebook_mode(connected=True)

def get_quandl_data(quandl_id):
    '''Download and cache Quandl dataseries'''
    cache_path = '{}.pkl'.format(quandl_id.replace('/', '-'))
    try:
        f = open(cache_path, 'rb')
        df = pickle.load(f)
        print('Loaded {} from cache'.format(quandl_id))
    except (OSError, IOError) as e:
        print('Downloading {} from Quandl'.format(quandl_id))
        df = quandl.get(quandl_id, returns="pandas")
        df.to_pickle(cache_path)
        print('Cached {} at {}'.format(quandl_id, cache_path))
    return df
```

Data Extraction – Live...

➤ Setting up tweepy stream:

```
listener = PrintListener()
keyToSearch = 'Bitcoin'
auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_token_secret)
stream = tweepy.Stream(auth, listener, timeout=30)
stream.filter(track=[keyToSearch])
```

➤ Defining call back:

```
class PrintListener(tweepy.StreamListener):
    def __init__(self, time_limit=10):
        self.start_time = time.time()
        self.limit = time_limit
        self.saveFile = open('Crytp.csv', 'a')
        super(PrintListener, self).__init__()

    def on_data(self, data):
        return True

    def on_error(self, status):
        print(status)

    def get_tweet_sentiment(self, tweet):
        return 1

    def clean_tweet(self, tweet):
        return True
```

Data Purging

- Cleaning up tweets – remove unwanted recurring words from Tweets

```
' '.join(re.sub("[^[:alnum:][:space:]]$", "", tweet.text)).replace('ampamp', '').replace('retweet', '').replace('just', '')\
.replace('comment', '').replace('amp', '').replace('bitcoin', '').replace('btc', '').replace('xrp', '').replace('xrp', '').replace('crypto'
, '')\
.replace('cryptocurrency', '').strip()
```

```
' '.join(re.sub("(@[A-Za-z0-9]+)|([^0-9A-Za-z \t])|(\w+:\ / \ / \S+)", " ", tweet).split())
```

Data Storage

- Storing the data to train and test model

```
import csv
csvFile = open('Demo.csv', 'a', newline='')
csvWriter = csv.writer(csvFile)
csvWriter.writerow([, ])
```

```
import pandas as pd
df = pd.DataFrame([])
df.to_csv(r'Demo.csv')
pd.read_csv(r'Demo.csv')
```

Sentiment Analysis - TextBlob

```
import tweepy
import re
import textblob as TextBlob
import csv

consumer_key = '#####'
consumer_secret = '#####'
access_token = '#####'
access_token_secret = '#####'
parsed_tweet = {}
tweets = []
dataParsedList = []

def clean_tweets(tweetss):
    return ' '.join(re.sub("@[A-Za-z0-9]+)|([^0-9A-Za-z \t])| (\w+:\ / \ / \S+)", " ", tweetss).split())

def remove_noise(tweet):
    return ''.join(re.sub("[^[:alnum:][:space:]]$", "", tweet.text)).replace('ampamp', '').replace('retweet', '').replace('just', '')\
        .replace('comment', '').replace('amp', '').replace('bitcoin', '').replace('btc', '').replace('xrp', '').replace('xrp', '').replace('crypto', '')\
        .replace('cryptocurrency', '').strip()

if __name__ == '__main__':
    auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
    auth.set_access_token(access_token, access_token_secret)
    csvFile = open('HistoricalTweets.csv', 'a', newline='')
    api = tweepy.API(auth, wait_on_rate_limit=True)
    csvWriter = csv.writer(csvFile)
    for tweet in tweepy.Cursor(api.search, q="#Bitcoin", count=100,
                               lang="en",
                               since="2018-08-13", until="2018-08-14").items():
        clean_tweet = remove_noise(tweet)
        clean_tweet = clean_tweets(clean_tweet)

        polarity = TextBlob.TextBlob(clean_tweet).sentiment.polarity
        if polarity != 0:
            csvWriter.writerow([tweet.created_at, clean_tweet, polarity])
```

Sentiment Analysis - NLTK

```
import tweepy
import re
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import csv

consumer_key = '#####'
consumer_secret = '#####'
access_token = '#####'
access_token_secret = '#####'
tweets = []
dataParsedList = []
sid = SentimentIntensityAnalyzer()

def vadel_sentiment(text):
    return sid.polarity_scores(text)['compound']

def clean_tweets(tweetss):
    return ' '.join(re.sub("(@[A-Za-z0-9]+)|([^0-9A-Za-z \t])|(\w+:\ / \ / \S+)", " ", tweetss).split())

def remove_noise(tweet):
    return ' '.join(re.sub("[^[:alnum:][:space:]]$", "", tweet.text)).replace('ampamp', '').replace('retweet', '').replace('just', '')\
        .replace('comment', '').replace('amp', '').replace('bitcoin', '').replace('btc', '').replace('xrp', '').replace('xrp', '').replace('crypto', '')\
        .replace('cryptocurrency', '').strip()

if __name__ == '__main__':
    auth = tweepy.OAuthHandler(consumer_key, consumer_secret)
    auth.set_access_token(access_token, access_token_secret)
    csvFile = open('HistoricalTweets.csv', 'a', newline='')
    api = tweepy.API(auth, wait_on_rate_limit=True)
    csvWriter = csv.writer(csvFile)
    for tweet in tweepy.Cursor(api.search, q="#Bitcoin", count=100,
                               lang="en",
                               since="2018-08-13", until="2018-08-14").items():
        clean_tweet = remove_noise(tweet)
        clean_tweet = clean_tweets(clean_tweet)
        polarity = vadel_sentiment(clean_tweet)
        if polarity != 0:
            csvWriter.writerow([tweet.created_at, clean_tweet, polarity])
```

Sentiment Analysis - NLTK

Tweet	TextBlob	NLTK
RT And you wonder why people hate Bitcoin BTC https t co D24QvAjn95	-0.8	-0.5719
Sentiment is super bullish today We re up like 15 after being 90 down I m still waiting for lower lows and true https t co 6zRrIJ376w	0.175925926	0.7351
Be greedy when others are fearful and fearful when others are greedy If you didn t sell in Dec 2017 I m sorry 4u https t co VEhtq8zLa6	-0.766666667	-0.8834
RT If It s Too Good To Be True Beware of Scams GAMB Crypto Bitcoin Blockcha	0.525	0.2263
The SRXIO token is very unique generating a monthly passive income regardless of whether the price of Bitcoin goes https t co E9fy0vpFjb	0.4875	0.2023

Tweet	TextBlob	NLTK
Bitcoin allows us to practice our greed We all know that it will be worth much more in 10 years	0.4	-0.2023
Look at that holochain sell wall coming down Crazy to think only 10 satoshi s is 100 gain still boggles my min	-0.251852	0.25
RT New sticker coming soon to the BlockstreamStore Guess what it is LightningNetwork reckless Bitcoin http	0.1363636	-0.4019

Sentiment Analysis - sklearn

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import random
data = []
#Train
data_labels = []
with open("./pos.txt") as f:
    for i in f:
        data.append(i)
        data_labels.append('pos')
with open("./neg.txt") as f:
    for i in f:
        data.append(i)
        data_labels.append('neg')

#Vectorizing
vectorizer = CountVectorizer(
    analyzer='word',
    lowercase=False,
)

#Fit and transform
#Centr the data =>  $x' = \frac{x-\mu}{\sigma}$ 
#Apply same to test data

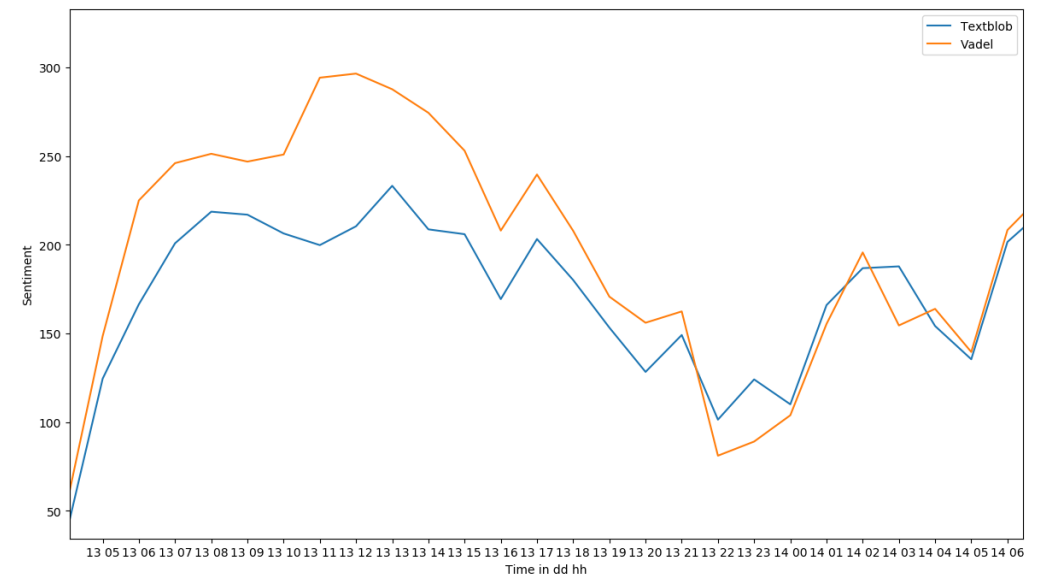
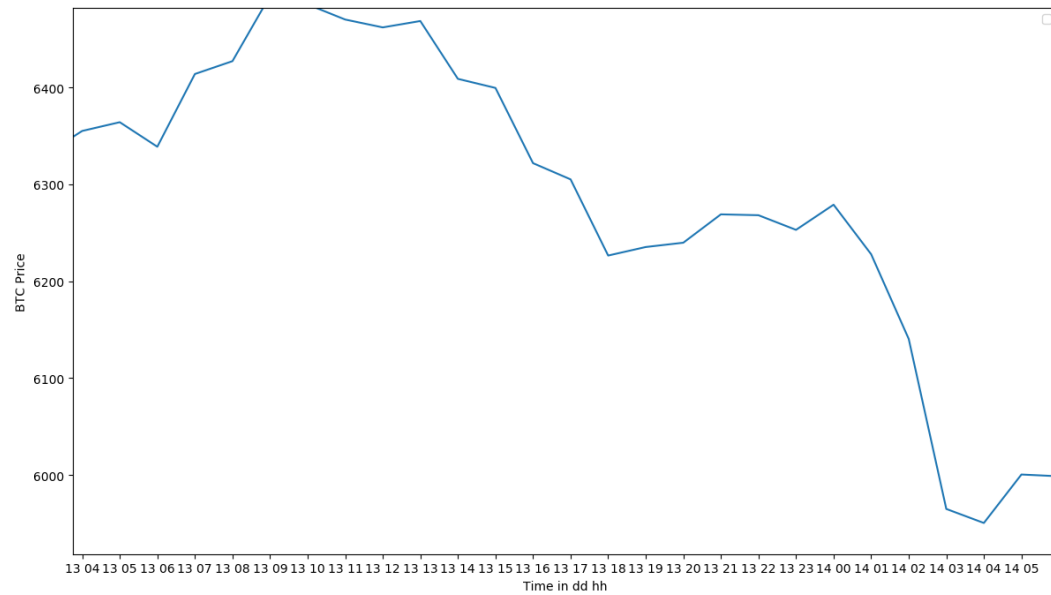
features = vectorizer.fit_transform(data)
features_nd = features.toarray()
X_train, X_test, y_train, y_test = train_test_split(features_nd, data_labels, train_size=0.80, random_state=1234)

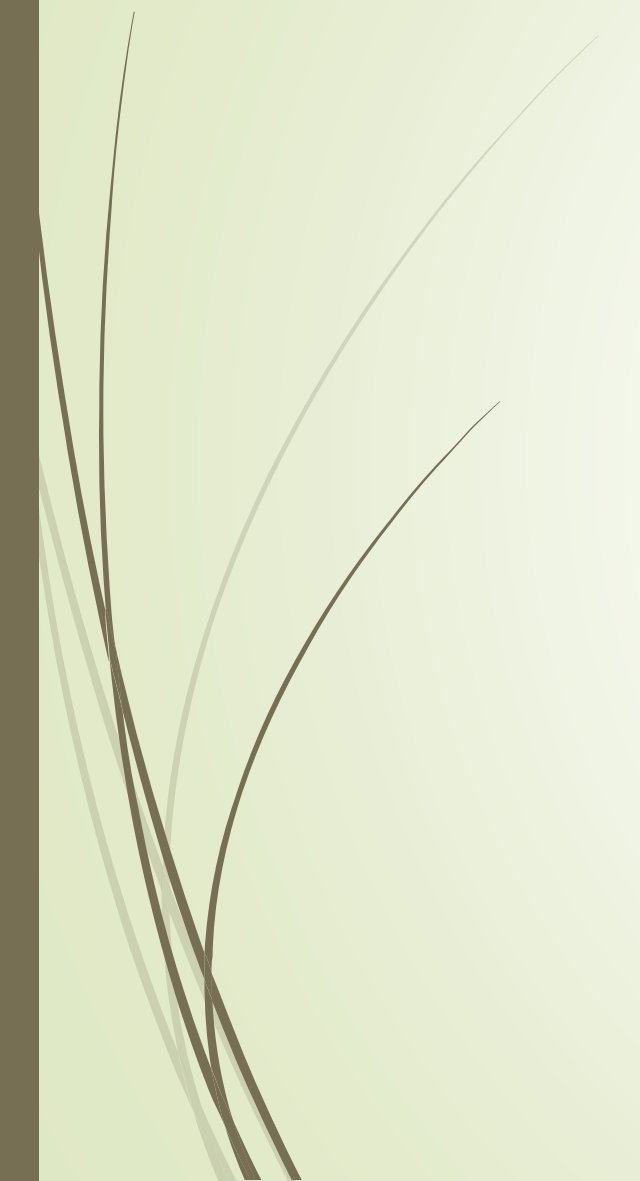
#Linear regression
log_model = LogisticRegression()

log_model = log_model.fit(X=X_train, y=y_train)
y_pred = log_model.predict(X_test)

j = random.randint(0, len(X_test)-7)
for i in range(j, j+7):
    print(y_pred[i])
    ind = features_nd.tolist().index(X_test[i].tolist())
    print(data[ind].strip())
print(accuracy_score(y_test, y_pred))
```

Data Analysis





Thank You